

First Principles

Why do we need auto scaling if we can just use a bigger instance?

Elasticity vs vertical scaling limits. Why limit yourself to just one big instance with a single point of failure if it cannot take the workload? When you split the work it relieves intense resource usage and makes the workload more balanced when you spread it out and in turn it will make it more efficient and therefore more cost effective.

What problem does EC2 solve at its core

EC2 solves compute. Whatever kind of workload you want to perform, from small test web servers to beefy HPC work, EC2 has you covered.

Why EC2 is insufficient for production traffic

When your production traffic is demanding, one EC2 by itself will not be resilient to push through. A lot of issues here: single point of failure, no high availability, no proper load balancing. Recipe for disaster. This is when things like ASGs, ALBs, and Multi-AZ deployments come in.

What ASG guarantees

ASG guarantees uniform deployment of EC2 instances by using a Launch Template, additionally it ensures high availability, elasticity, and scalability by allowing you to configure Multi-AZ, creating and deleting instances based on desired capacity (min and max instances as well).

What ASG does not guarantee

Anything outside the ASG. For example, the standalone webserver which I created the Launch Template off of; when I deleted it, the ASG did not perform any action.

I guess what GPT wants me to know is that ASG's guarantee quantity, not quality. It does not guarantee my application works, it does not guarantee traffic is being served, it does not guarantee zero downtime, it does not guarantee performance, and it does not protect you from bad decisions.

What's the real problem load balancers solve?

Load balancers ensure that instances are highly available and offer the option to have zero-downtime deployments, as well as proportionally distributing traffic how you want it.